



॥ नवोन्मेषेण देशनिर्माणम् ॥

భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

The Paninian Pratyāhāra Engine

A Computational Implementation of Pratyāhāra Formation
from the Māheśvara (Śiva) Sūtras

Sub-module of the Telugu Paninian Rule Engine Project
(Mapping Bāla Vyākaraṇam rules onto Pāṇini's Aṣṭādhyāyī)

Prepared by: Teja Vardhan (ee24btech11034@iith.ac.in)
Cherisha (es24btech11003@iith.ac.in)

Under the guidance of: Lakshmi Valli K
(manager.outreach@hst.iith.ac.in)

Institution: Indian Institute of Technology Hyderabad

September 25, 2026

Contents

1	Introduction and Motivation	2
2	The Fourteen Śiva Sūtras	2
3	Data Representation and Linearisation	3
4	The PratyaharaEngine Class	3
4.1	Locating the starting phoneme	4
4.2	Locating the terminating anubandha	4
4.3	Extracting the phoneme class	4
4.4	Validation and bulk enumeration	5
4.5	Combining with an external sound set	6
5	Verified Behaviour	6
6	Why This Approach, and Not the Alternatives	6
6.1	Alternative: a hard-coded lookup table	6
6.2	Alternative: regular expressions over a transliterated string	7
6.3	Why the chosen design is preferable	7
7	Status and Next Steps	7

1 Introduction and Motivation

Pāṇini's Aṣṭādhyāyī refers to groups of sounds constantly throughout its rules – for example “all vowels”, “all consonants”, or “all sounds from इ to ल्”. Rather than naming each group in full every time, Pāṇini encodes them once, at the very start of the grammar, in the fourteen Māheśvara Sūtras (commonly called the Śiva Sūtras). Every subsequent rule in the Aṣṭādhyāyī can then refer to a sound-class in a single short syllable – a **pratyāhāra** – such as अण्, इक्, or हल्.

This compression is only possible because of a fixed convention: a pratyāhāra name is formed by a starting phoneme followed by a marker sound (anubandha, or “it”) that occurs later in the sequence. The class denoted by the pratyāhāra is every ordinary phoneme lying between the two – the marker itself is silent and never part of the class.

This document describes a small Python engine that mechanises this convention: given the fourteen sūtras and their it-markers, it computes the phoneme set denoted by any valid pratyāhāra notation, and can enumerate every pratyāhāra the sūtras make possible. This engine is being built as a reusable component of the larger Telugu Paninian Rule Engine project, where Pāṇinian sound-classes are used as building blocks for encoding sandhi and sañjñā rules from the Bāla Vyākaraṇam.

The rest of this document is organised so that each code block is immediately followed by an explanation of what it does and why it is written that way – it is meant to be read top to bottom without needing to jump between sections of the source file.

2 The Fourteen Śiva Sūtras

Table 1 lists the sūtras exactly as they appear in the grammar. Each row is one sūtra; the final, underlined sound in each row is its anubandha (it-marker) and is **not** itself part of any pratyāhāra – it only marks where a pratyāhāra beginning earlier may end.

Sūtra #	Phonemes (final sound underlined = anubandha)
1	अ इ उ ण्
2	ऋ लृ क्
3	ए ओ ङ्
4	ऐ औ च्
5	ह्रस्व र्त्
6	लृ ण्
7	ज् म् ङ् ण् न् म्
8	झ भ् ज्
9	घ ढ ध् ष्
10	ज् ब् ग् ङ् द श्
11	ख् फ् छ् ठ् थ् च्
12	क् प् य्
13	श् ष् स् र्
14	हल्

Sūtra # Phonemes (final sound underlined = anubandha)

Table 1: The fourteen Śiva Sūtras. 40 distinct ordinary phonemes and 14 anubandha occurrences, concatenated in this order, form one linear sequence of 54 symbols.

The engine's very first job is to turn this table into exactly that linear sequence, since a pratyāhāra is defined purely in terms of position within it.

3 Data Representation and Linearisation

Each sūtra is represented as a list of (symbol, is_anubandha) pairs, in the order Pāṇini gave them, with True marking an it. This is the only place the raw grammar text appears in the program – everything downstream is generic logic that works over any such list, not over Devanāgarī text specifically.

```
SHIVA_SUTRAS = [
    [(अ, False), (इ, False), (उ, False), (ए, True)],
    [(ऋ, False), (ॠ, False), (क्, True)],
    # ... remaining 12 sutras, same (symbol, is_anubandha) pattern
]
```

The sūtras are then flattened into one continuous sequence, and each entry is tagged with the sūtra number and position it came from – useful for debugging, though not used by the core algorithm itself:

```
1 SEQUENCE = []
2 for sutra_number, sutra in enumerate(SHIVA_SUTRAS, start=1):
3     for position, (symbol, is_anubandha) in enumerate(sutra):
4         SEQUENCE.append({
5             "symbol": symbol,
6             "is_anubandha": is_anubandha,
7             "sutra": sutra_number,
8             "position": position,
9         })
```

SEQUENCE is now a flat list of 54 dictionaries – 40 ordinary phonemes and 14 anubandhas, in sūtra order. This single list is what every method below operates on. Reducing the whole grammar table to one flat, indexable list is the key design decision of this engine: it turns a linguistic rule (“everything between a sound and its later marker”) into a plain array-slicing problem.

4 The PratyaharaEngine Class

The engine wraps SEQUENCE and precomputes two convenience lists: all ordinary phonemes (self.sounds) and all anubandhas (self.anubandhas), each used later for validation and notation parsing.

```
1 class PratyaharaEngine:
2     def __init__(self, sequence):
```

```

3     self.sequence = sequence
4     self.sounds = [i["symbol"] for i in sequence if not i["is_anubandha"]]]
5     self.anubandhas = [i["symbol"] for i in sequence if i["is_anubandha"]]]

```

4.1 Locating the starting phoneme

```

1 def find_start(self, start):
2     for index, item in enumerate(self.sequence):
3         if item["symbol"] == start and not item["is_anubandha"]:
4             return index
5     raise ValueError(f"Starting sound '{start}' is not present in the Shiva Sutas.")

```

This performs a linear scan for the first occurrence of `start` as an ordinary phoneme. A scan is enough here: `SEQUENCE` has only 54 entries, so there is no performance reason to build an index, and a plain scan keeps the logic transparent and easy to verify against Table 1 by eye.

4.2 Locating the terminating anubandha

```

1 def find_end(self, start_index, ending_marker):
2     for index in range(start_index + 1, len(self.sequence)):
3         item = self.sequence[index]
4         if item["symbol"] == ending_marker and item["is_anubandha"]:
5             return index
6     raise ValueError(f"Anubandha '{ending_marker}' does not occur after the starting sound.")

```

This search deliberately starts after `start_index`, not from the beginning of the sequence. That single choice encodes Pāṇini's own constraint: a pratyāhāra's marker must occur later in the sūtra order than its starting phoneme – a marker occurring earlier cannot close it. Enforcing this directionally, rather than searching the whole list and picking the nearest match, is what stops the engine from silently accepting an invalid pratyāhāra name.

4.3 Extracting the phoneme class

```

1 def get_pratyahara(self, notation, ending_marker=None):
2     if ending_marker is not None:
3         start = notation
4     else:
5         start, ending_marker = None, None
6         for candidate in sorted(self.sounds, key=len, reverse=True):
7             if notation.startswith(candidate):
8                 remaining = notation[len(candidate):]
9                 if remaining in self.anubandhas:
10                    start, ending_marker = candidate, remaining
11                    break
12     if start is None:
13         raise ValueError(f"Could not parse '{notation}'.")

```

```

14
15     start_index = self.find_start(start)
16     end_index = self.find_end(start_index, ending_marker)
17     return [item["symbol"] for item in self.sequence[start_index:end_index]
18             if not item["is_anubandha"]]

```

This method accepts two calling styles: `get_pratyahara("R", "sh")` when the caller already knows where the start ends and the marker begins, or `get_pratyahara("Rsh")` when only the combined notation string is known and it must be split automatically.

The split is done by trying every known phoneme as a candidate starting sound, **longest first**, and checking whether the leftover suffix is a known anubandha. Longest-first matching matters specifically for Devanāgarī: several phonemes in the sūtras are themselves multi-code-point clusters (a consonant plus a virama, or a vowel sign), so a shorter candidate can be a prefix of a longer one. Trying the longest candidates first guarantees the split lands on the correct phoneme boundary rather than an accidental partial match.

Once `start` and `ending_marker` are known, extraction is a single slice of `SEQUENCE` with the anubandha entries filtered out – exactly the definition of a pratyāhāra given in Section 1.

4.4 Validation and bulk enumeration

```

1  def is_valid(self, notation, ending_marker=None):
2      try:
3          self.get_pratyahara(notation, ending_marker)
4          return True
5      except ValueError:
6          return False
7
8  def generate_all(self):
9      results = []
10     for start_index, start_item in enumerate(self.sequence):
11         if start_item["is_anubandha"]:
12             continue
13         start = start_item["symbol"]
14         for end_index in range(start_index + 1, len(self.sequence)):
15             end_item = self.sequence[end_index]
16             if not end_item["is_anubandha"]:
17                 continue
18             sounds = [i["symbol"] for i in self.sequence[start_index:
19                                                             end_index]
20                     if not i["is_anubandha"]]
21             results.append({
22                 "notation": start + end_item["symbol"],
23                 "start": start,
24                 "ending_marker": end_item["symbol"],
25                 "sounds": sounds,
26             })
27     return results

```

`is_valid` simply re-uses `get_pratyahara` and converts a raised error into a boolean, so there is only one place that defines what “valid” means. `generate_all` pairs every ordinary phoneme

with every anubandha that appears after it – an $O(n^2)$ enumeration, but n is 54, so this runs instantly and is useful for producing a complete reference table (Section 6) or for regression-testing changes to the sūtra data.

4.5 Combining with an external sound set

```
1 def filter_with_array(self, notation, allowed_sounds):
2     return [s for s in self.get_pratyahara(notation) if s in allowed_sounds
            ]
```

This is a convenience for the wider rule-engine project: a grammar rule is often stated as “the pratyāhāra हल्, restricted to the sounds relevant to this rule’s context”. Rather than re-deriving that restriction ad hoc, callers elsewhere in the project can intersect any pratyāhāra with any other list of sounds through this one method.

5 Verified Behaviour

Running the engine against the sūtras in Table 1 confirms it reproduces the pratyāhāras exactly as given in the traditional grammar:

```
get_pratyahara("अण्") -> अ, इ, उ (3 sounds)
get_pratyahara("इक्") -> इ, उ, ऋ, लृ (4 sounds)
get_pratyahara("ऋ", "ष्") -> ऋ, लृ, ए, ओ, ऐ, औ, ह्र, य, व, र, ...
generate_all() -> 293 valid pratyaharas over 40 sounds / 14 markers
```

One genuine edge case surfaced during testing: the classical name हल् (“all consonants”) is traditionally written with the starting consonant unmarked, but `self.sounds` stores every consonant with its inherent virama (ह्र, not bare ह). As written, `get_pratyahara` therefore requires the fully-marked form ह्र rather than the conventional हल्. This does not affect `get_pratyahara(start, ending_marker)` called with two explicit arguments, and is flagged here as a known limitation to resolve before the notation parser is exposed to end users (see Section 7).

6 Why This Approach, and Not the Alternatives

Several simpler ways to get from “pratyāhāra name” to “list of sounds” were considered before settling on this design.

6.1 Alternative: a hard-coded lookup table

The most obvious alternative is a dictionary literal mapping every pratyāhāra name directly to its sound list – e.g. `{"aN": ["a", "i", "u"], "ik": ["i", "u", "R", "L"], ...}`. This was rejected because:

- There are 293 valid pratyāhāras derivable from the fourteen sūtras (Section 6); hand transcribing all of them is tedious and error-prone, and any transcription mistake is silent – it produces no error, just a wrong sound list.

- It has no relationship to the source text. A reviewer checking the code against the actual Śiva Sūtras has no way to confirm correctness other than trusting the table.
- It cannot adapt if the underlying phoneme inventory changes – for instance, if the wider Telugu project later needs additional it-markers or a modified sūtra ordering. A lookup table would need to be regenerated by hand; the engine only needs SHIVA_SUTRAS edited.

6.2 Alternative: regular expressions over a transliterated string

Concatenating all sūtras into one string and using regex to slice out substrings was also considered. This is fragile specifically because of the script involved: Devanāgarī consonants combine with viramas and vowel signs as multi-code-point clusters, so byte- or character-offset slicing does not reliably align with phoneme boundaries. The token-list model used here (Section 3) sidesteps this completely – every element of SEQUENCE is already one whole phoneme, so no string-boundary logic is needed anywhere in the engine.

6.3 Why the chosen design is preferable

Encoding the sūtras once as data and deriving every pratyāhāra algorithmically has three concrete advantages over both alternatives:

1. **Traceability.** SHIVA_SUTRAS is a direct, checkable transcription of the source grammar (Table 1). Anyone auditing the code compares 14 short lists against 14 sūtras, not 293 entries against a reference table.
2. **Correctness by construction.** The rule for forming a pratyāhāra – “everything between a phoneme and a later marker” – is implemented once (`find_start` / `find_end` / the slice in `get_pratyahara`) and applies uniformly to all 293 combinations, so it is impossible for one entry to follow the rule and another not to.
3. **Reusability inside the larger project.** Because the engine exposes a class with general methods (`parse`, `validate`, `enumerate`, `filter`) rather than a fixed table, the same module can be imported wherever the Bāla Vyākaraṇam rule-mapping project needs a Pāṇinian sound-class – including `filter_with_array`, which lets a sandhi or sañjñā rule elsewhere in the project intersect a pratyāhāra with its own context-specific sound set without re-implementing this logic.

The trade-off is a small amount of up-front logic (roughly 40 lines) compared to a flat table, and the notation-splitting edge case noted in Section 5. Both are one-time costs; the lookup table’s costs (293 manually verified entries, no traceability, no adaptability) would recur every time the underlying grammar data changed.

7 Status and Next Steps

- **Done:** sūtra data encoded and linearised; core engine (`find_start`, `find_end`, `get_pratyahara`, `is_valid`, `generate_all`, `filter_with_array`) implemented and verified against all 293 derivable pratyāhāras.

- **Open issue:** resolve the bare-consonant notation case (हल् vs. ह्र्) described in Section 5, so that classical pratyāhāra names can be typed exactly as written in the grammar.
- **Next:** integrate this engine as the sound-class layer beneath the sañjñā and sandhi prakaraṇam rule definitions of the Telugu Paninian Rule Engine, so that those rules can refer to Pāṇinian pratyāhāras directly instead of listing sounds by hand.